Année 2025-2026

Mentor : Mr Arnol Namegni

Projet 3

Entraînez-vous avec SQL et créez votre BDD

Collecter - Analyser - Répondre aux besoins

Candidat : Mr Jean-Francois Chaussoy

Alternant Data Engineering

Contexte Projet & Environnement Technique

Mission :
Laplace Immo est un réseau national d'agences immobilières qui veut exploiter ses données pour mieux comprendre le marché.

Objectif :
construire un Proof of Concept (POC) pour centraliser les transactions immobilières du 1er semestre 2020, afin de :
- Suivre l'évolution des prix au mètre carré
- Identifier les régions les plus porteuses
- Fournir des indicateurs clés à la direction pour piloter la stratégie commerciale

Problématique business :
Comment valider la pertinence de l'offre de l'entreprise, vis à vis du territoire et de l'évolution du marché immobilier ?

Stratégie de sauvegarde et conformité RGPD

RGPD:

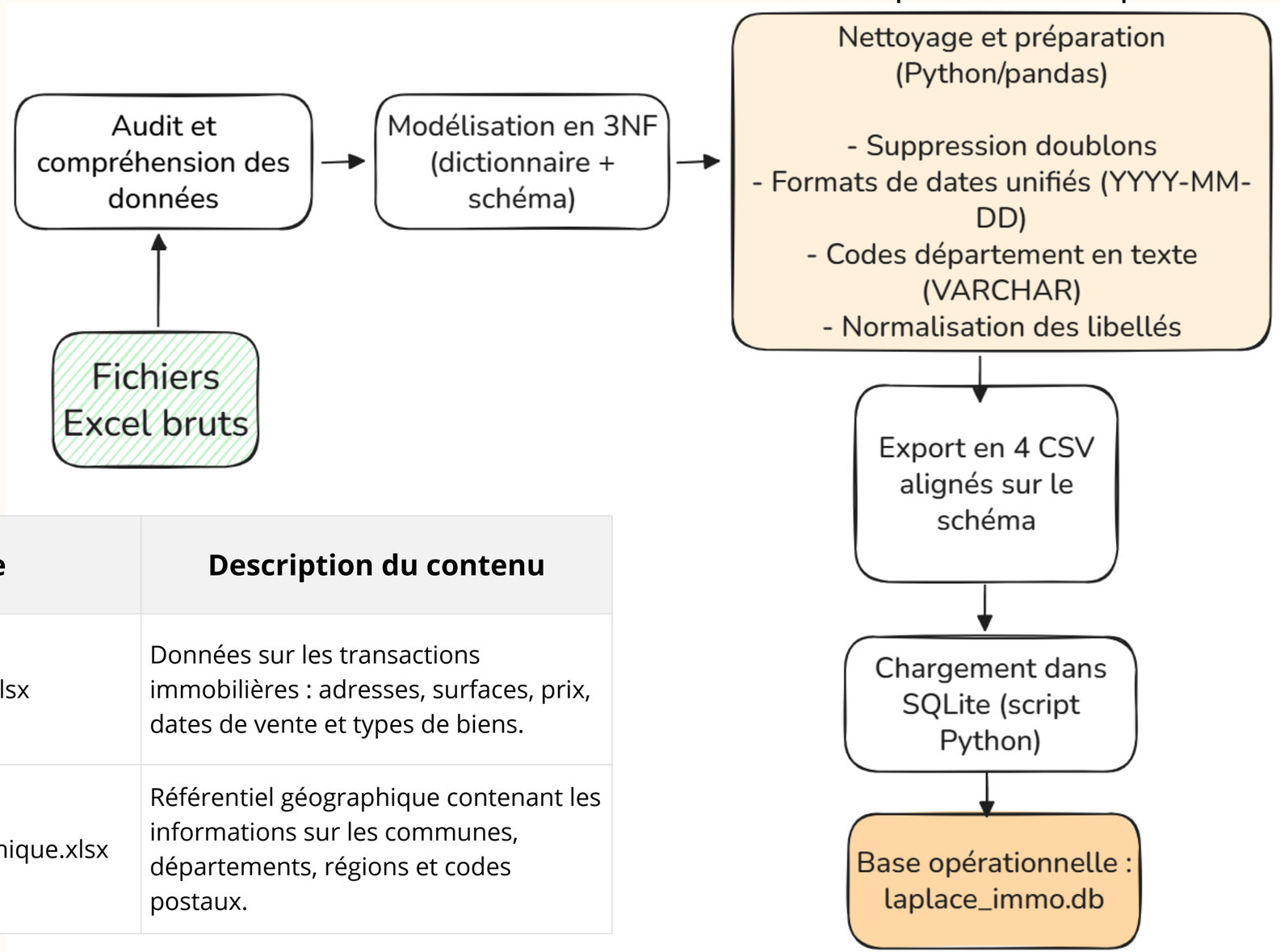
- Seules les données strictement nécessaires à l'analyse immobilière sont conservées
- Aucune donnée personnelle sensible (pas de noms de propriétaires, pas d'informations bancaires)
- Les adresses sont conservées uniquement pour la localisation géographique des biens

Sauvegarde:

- Le code source et les scripts sont versionnés avec Git (GitHub)
- La base de données SQLite est un fichier portable, facile à sauvegarder et à dupliquer
- Les données sources (CSV) sont conservées dans le dépôt pour permettre une reconstruction complète

Des données brutes à la base opérationnelle

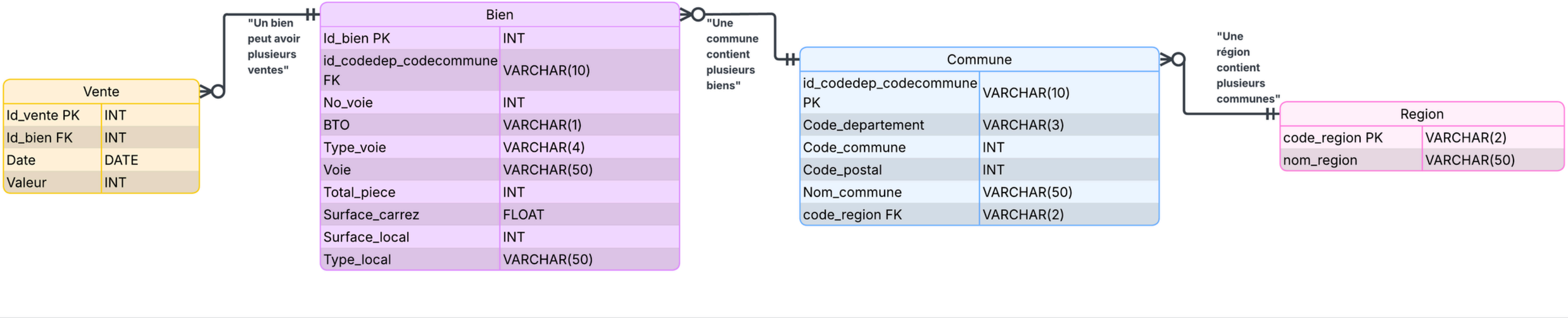
Le passage des fichiers Excel bruts à la base de données s'est fait en plusieurs étapes :



Fichier source	Description du contenu
Valeurs-foncières.xlsx	Données sur les transactions immobilières : adresses, surfaces, prix, dates de vente et types de biens.
fr-esr-referentiel-geographique.xlsx	Référentiel géographique contenant les informations sur les communes, départements, régions et codes postaux.

Schema Relationnel

Schema relationnel - Projet Laplace Immo



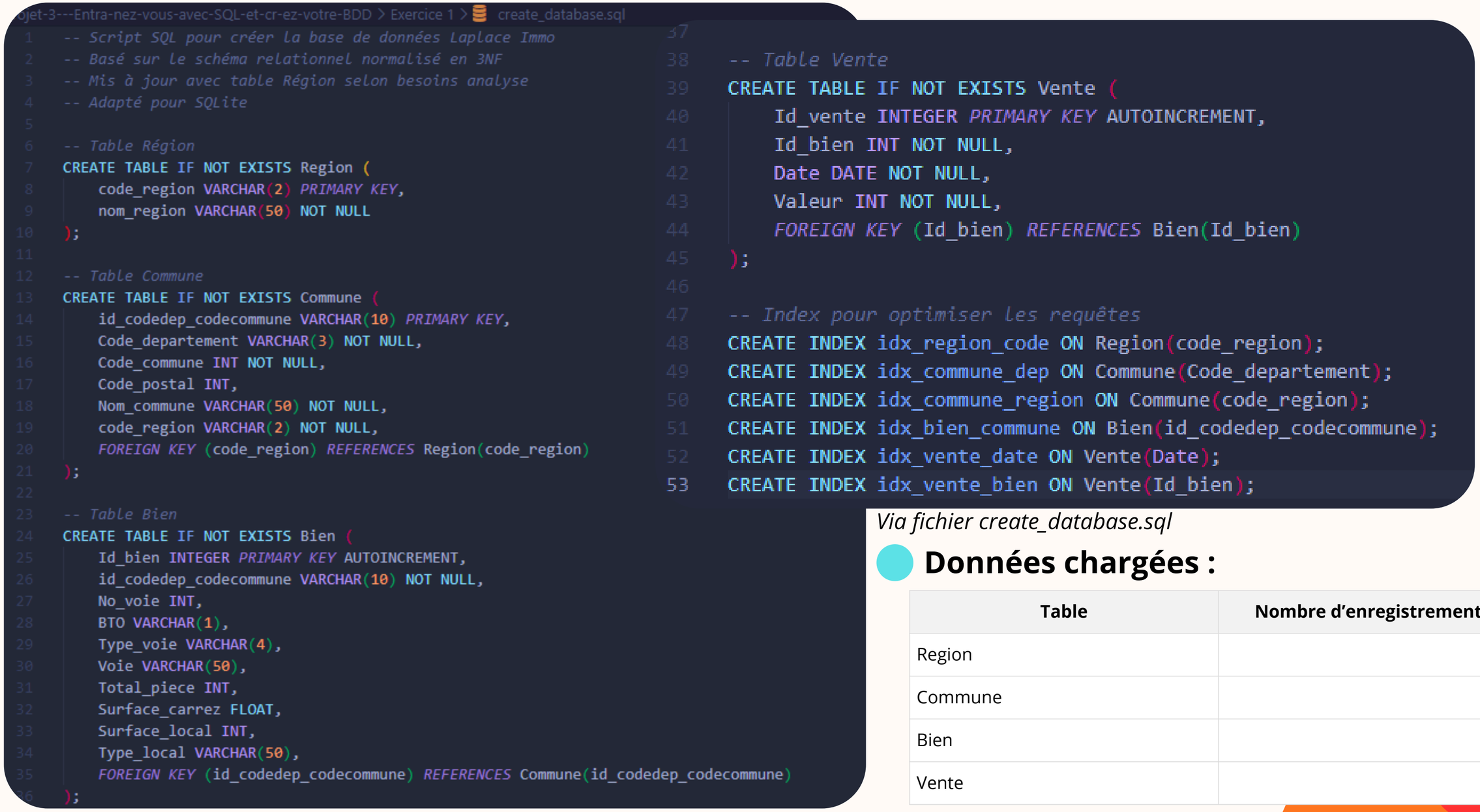
Via lucid.app

Pourquoi la 3NF ? :

La 3ème forme normale garantit qu'il n'y a pas de redondance dans les données. Chaque information n'est stockée qu'une seule fois :

- Le nom d'une région n'apparaît que dans la table Region (pas répété dans chaque commune)
- Les infos d'une commune ne sont pas copiées dans chaque bien
- Les relations entre tables se font via des **clés étrangères** (FK)

La base de données : tables créées et données chargées



```sql
-- Script SQL pour créer la base de données Laplace Immo
-- Basé sur le schéma relationnel normalisé en 3NF
-- Mis à jour avec table Région selon besoins analyse
-- Adapté pour SQLite

-- Table Région
CREATE TABLE IF NOT EXISTS Region (
    code_region VARCHAR(2) PRIMARY KEY,
    nom_region VARCHAR(50) NOT NULL
);

-- Table Commune
CREATE TABLE IF NOT EXISTS Commune (
    id_codedep_codecommune VARCHAR(10) PRIMARY KEY,
    Code_departement VARCHAR(3) NOT NULL,
    Code_commune INT NOT NULL,
    Code_postal INT,
    Nom_commune VARCHAR(50) NOT NULL,
    code_region VARCHAR(2) NOT NULL,
    FOREIGN KEY (code_region) REFERENCES Region(code_region)
);

-- Table Bien
CREATE TABLE IF NOT EXISTS Bien (
    Id_bien INTEGER PRIMARY KEY AUTOINCREMENT,
    id_codedep_codecommune VARCHAR(10) NOT NULL,
    No_voie INT,
    BTO VARCHAR(1),
    Type_voie VARCHAR(4),
    Voie VARCHAR(50),
    Total_piece INT,
    Surface_carrez FLOAT,
    Surface_local INT,
    Type_local VARCHAR(50),
    FOREIGN KEY (id_codedep_codecommune) REFERENCES Commune(id_codedep_codecommune)
);

-- Table Vente
CREATE TABLE IF NOT EXISTS Vente (
    Id_vente INTEGER PRIMARY KEY AUTOINCREMENT,
    Id_bien INT NOT NULL,
    Date DATE NOT NULL,
    Valeur INT NOT NULL,
    FOREIGN KEY (Id_bien) REFERENCES Bien(Id_bien)
);

-- Index pour optimiser les requêtes
CREATE INDEX idx_region_code ON Region(code_region);
CREATE INDEX idx_commune_dep ON Commune(Code_departement);
CREATE INDEX idx_commune_region ON Commune(code_region);
CREATE INDEX idx_bien_commune ON Bien(id_codedep_codecommune);
CREATE INDEX idx_vente_date ON Vente(Date);
CREATE INDEX idx_vente_bien ON Vente(Id_bien);
```

Via fichier create_database.sql

Données chargées :

Table	Nombre d'enregistrements
Region	8
Commune	15
Bien	105
Vente	160

Vérification de la structure créée (PRAGMA)

Commande SQL :

Après l'exécution du script DDL, j'ai vérifié que la structure réelle de la base correspond au schéma défini :

```
PRAGMA table_info(Region);
PRAGMA table_info(Commune);
PRAGMA table_info(Bien);
PRAGMA table_info(Vente);
```

Table Region :

Colonne	Type de donnée	Obligatoire (NOT NULL)	Clé primaire
code_region	VARCHAR(2)	—	✓
nom_region	VARCHAR(50)	✓	—

Table Commune :

Colonne	Type de donnée	Obligatoire (NOT NULL)	Clé primaire
id_codedep_codecommune	VARCHAR(10)	—	✓
Code_departement	VARCHAR(3)	✓	—
Code_commune	INT	✓	—
Code_postal	INT	—	—
Nom_commune	VARCHAR(50)	✓	—
code_region	VARCHAR(2)	✓	—

Vérification de la structure créée (PRAGMA)

Table Bien :

Colonne	Type de donnée	Obligatoire (NOT NULL)	Clé primaire
Id_bien	INTEGER	—	✓ (auto-incrémentée)
id_codedep_codecommune	VARCHAR(10)	✓	—
No_voie	INT	—	—
BTO	VARCHAR(1)	—	—
Type_voie	VARCHAR(4)	—	—
Voie	VARCHAR(50)	—	—
Total_piece	INT	—	—
Surface_carrez	FLOAT	—	—
Surface_local	INT	—	—
Type_local	VARCHAR(50)	—	—

Table Vente :

Colonne	Type de donnée	Obligatoire (NOT NULL)	Clé primaire
Id_vente	INTEGER	—	✓ (auto-incrémentée)
Id_bien	INT	✓	—
Date	DATE	✓	—
Valeur	INT	✓	—

La structure correspond exactement au dictionnaire des données et au schéma relationnel.

Requête 1 — Total d'appartements vendus au S1 2020

Requête SQL :

```
SELECT COUNT(*) as nombre_total_appartements_vendus
FROM Vente v
JOIN Bien b ON v.Id_bien = b.Id_bien
WHERE b.Type_local = 'Appartement'
 AND strftime('%Y', v.Date) = '2020'
 AND strftime('%m', v.Date) BETWEEN '01' AND '06';
```

Résultat:

31 appartements vendus

Compétences SQL :

`JOIN`, `WHERE`, `COUNT`, `strftime`

Requête 2 — Ventes d'appartement par région

Requête SQL :

```
SELECT r.nom_region,
    COUNT(*) as nombre_ventes_appartements
FROM Vente v
JOIN Bien b ON v.Id_bien = b.Id_bien
JOIN Commune c ON b.id_codedep_codecommune = c.id_codedep_codecommune
JOIN Region r ON c.code_region = r.code_region
WHERE b.Type_local = 'Appartement'
 AND strftime('%Y', v.Date) = '2020'
 AND strftime('%m', v.Date) BETWEEN '01' AND '06'
GROUP BY r.nom_region
ORDER BY nombre_ventes_appartements DESC;
```

Top Régions :

Région	Nombre de ventes
Pays de la Loire	8
Provence-Alpes-Côte d'Azur (PACA)	7
Île-de-France	6
Hauts-de-France	3

Requête 4 — Top 10 départements : prix au m²

- **Requête SQL :**

```
SELECT c.Code_departement,
    ROUND(AVG(v.Valeur / b.Surface_carrez), 2) as prix_moyen_m2
FROM Vente v
JOIN Bien b ON v.Id_bien = b.Id_bien
JOIN Commune c ON b.id_codedep_codecommune = c.id_codedep_codecommune
WHERE b.Type_local = 'Appartement'
 AND b.Surface_carrez > 0
GROUP BY c.Code_departement
ORDER BY prix_moyen_m2 DESC
LIMIT 10;
```

- **Top Départements :**

Classement (Top 3)	Prix moyen au m² (€)
67 — Bas-Rhin	10 455€
13 — Bouches-du-Rhône	8 485€
83 — Var	7 580€

Requête 7 — Évolution Q1 vs Q2 2020 (CTE)

Requête SQL :

```
WITH ventes_trimestrielles AS (
  SELECT
    CASE
      WHEN strftime('%m', v.Date) IN ('01','02','03') THEN 'Q1'
      WHEN strftime('%m', v.Date) IN ('04','05','06') THEN 'Q2'
    END as trimestre,
    COUNT(*) as nombre_ventes
  FROM Vente v
  WHERE strftime('%Y', v.Date) = '2020'
  GROUP BY trimestre)

SELECT q1.nombre_ventes as ventes_q1,
    q2.nombre_ventes as ventes_q2,
    ROUND(((q2.nombre_ventes - q1.nombre_ventes) * 100.0
      / q1.nombre_ventes), 2) as taux_evolution
FROM ventes_trimestrielles q1
CROSS JOIN ventes_trimestrielles q2
WHERE q1.trimestre = 'Q1' AND q2.trimestre = 'Q2';
```

Evolution du marché :

Trimestre	Nombre de ventes	Évolution
Q1	39	—
Q2	37	**-5,13%**

Compétences SQL :

`CTE (WITH)`, `CASE WHEN`, `CROSS JOIN`

Requête 11 — Top communes par département (Window Function)

Requête SQL :

```
WITH ventes_departements AS (
  SELECT c.Code_departement, c.Nom_commune,
      ROUND(AVG(v.Valeur), 2) as valeur_fonciere_moyenne,
      ROW_NUMBER() OVER (
        PARTITION BY c.Code_departement
        ORDER BY AVG(v.Valeur) DESC) as rang_commune
  FROM Vente v
  JOIN Bien b ON v.Id_bien = b.Id_bien
  JOIN Commune c ON b.id_codedep_codecommune = c.id_codedep_codecommune
  WHERE c.Code_departement IN ('06','13','33','59','69')
  GROUP BY c.Code_departement, c.id_codedep_codecommune, c.Nom_commune)

SELECT Code_departement, Nom_commune, valeur_fonciere_moyenne
FROM ventes_departements
WHERE rang_commune <= 3
ORDER BY Code_departement, rang_commune;
```

Top communes par départements :

Département	Commune	Valeur moyenne (€)
33 — Gironde	Bordeaux	689 838€
69 — Rhône	Lyon 2e	683 179€
13 — Bouches-du-Rhône	Marseille 2e	678 803€

Compétences SQL :

`ROW_NUMBER() OVER (PARTITION BY)`, sous-requête CTE

Synthèse des 12 requêtes

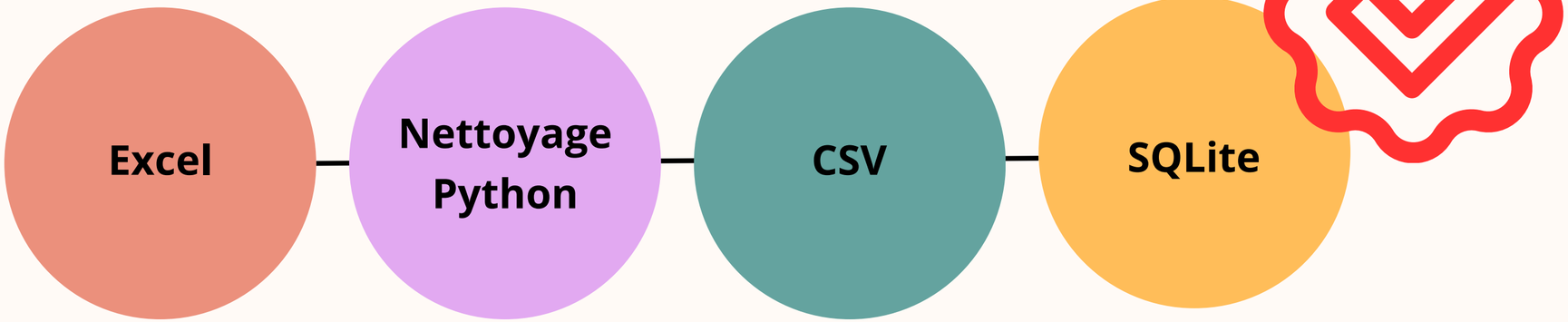
Rang	Besoin analysé	Résultat principal
1	Total d’appartements vendus au S1 2020	31 ventes
2	Répartition des ventes par région	Pays de la Loire en tête avec 8 ventes
3	Proportion des ventes par nombre de pièces	2 pièces et 6 pièces à 25,81 % chacun
4	Top 10 des départements par prix au m²	Bas-Rhin en tête avec 10 455 €/m²
5	Prix moyen au m² des maisons en Île-de-France	4 861 €/m²
6	Top 10 des appartements les plus chers	Nantes en tête avec 934 k€
7	Évolution entre le 1er et le 2e trimestre	Baisse de 5,13 %
8	Régions avec appartements de plus de 4 pièces	PACA en tête avec 13 345 €/m²
9	Communes avec au moins 50 ventes au 1er trimestre	Aucune (limitation liée au POC)
10	Écart de prix entre 2 pièces et 3 pièces	-23,65%
11	Top 3 des communes dans les départements ciblés	Bordeaux en tête avec 690 k€
12	Nombre de transactions pour 1 000 habitants	Non exploitable dans le cadre du POC

Conclusion

Ce POC valide la faisabilité technique :

Modélisation en 3NF robuste (4 tables, clés étrangères)

Pipeline complet :



12 besoins métier couverts par des requêtes SQL opérationnelles

Merci pour votre attention !